TrajData: On Vehicle Trajectory Collection With Commodity Plug-and-Play OBU Devices

Zhu Xiao, *Senior Member, IEEE*, Fancheng Li, Ronghui Wu, Hongbo Jiang, *Senior Member, IEEE*, Yupeng Hu, *Senior Member, IEEE*, Ju Ren, *Senior Member, IEEE*, Chenglin Cai, and Arun Iyengar, *Fellow, IEEE*

***Abstract*—For years, vehicle trajectory data have increasingly been important for a wide range of applications, from driver behavior investigation/classification, travel time/distance estimation, and routing in vehicular networks, to vehicle energy/emission evaluation. This article presents TrajData, the first systematic solution to reliable vehicle trajectory data collection, with only reliance on commercial-off-the-shelf (COTS) onboard unit (OBU) devices that utilize lightweight GPS modules and low-cost onboard diagnostics (OBD) readers. In the practical use of trajectory collection, GPS outages inevitably occur in urban environments thereby leading to large trajectory errors as well as missing vehicle location data. To resolve this, we propose a novel data-fusion-enabled deep learning approach with the purpose of achieving reliable vehicle trajectory collection in various urban road conditions. Specifically, we leverage motion information retrieved from OBD readers in TrajData to help reconstruct the trajectory data during GPS outages. By investigating the changes of direction angle from the OBD readings, we can identify different types of road sections. Furthermore, we integrate the neural arithmetic logic units (NALUs) into our trajectory reconstruction model to tame the challenges when GPS outages take place in various road sections. Experimental results from realistic data have demonstrated the effectiveness and reliability of the proposed method. In the road test, TrajData achieves an average position error below 15-m around a 60-s GPS outage, even in complex road sections, i.e., continuous turns and driving with accelerations/decelerations resulting in frequent changes of direction and speed.**

***Index Terms*—Internet of Vehicles (IoV), onboard unit (OBU) devices, private cars, trajectory collection.**

Manuscript received February 1, 2020; revised April 22, 2020 and May 25, 2020; accepted June 7, 2020. Date of publication June 11, 2020; date of current version September 15, 2020. This work was supported in part by the National Natural Science Foundation of China under Grant 61772184 and Grant U19A2067, in part by the Key Research and Development Project of Hunan Province of China under Grant 2018GK2014, in part by the Scientific Research Project of Department of the Natural Resources of Hunan Province under Grant 201910, in part by the Open Fund of State Key Laboratory of Geo Information Engineering under Grant SKLGIE2018-M-4-3, and in part by the Fund of Hubei Key Laboratory of Transportation Internet of Things under Grant WHUTIOT-2019004 and Grant 2018IOT004. *(Zhu Xiao and Hongbo Jiang contributed equally to this work.) (Corresponding authors: Ronghui Wu; Chenglin Cai.)*

Zhu Xiao, Fancheng Li, Ronghui Wu, Hongbo Jiang, and Yupeng Hu are with the College of Computer Science and Electronic Engineering, Hunan University, Changsha 410082, China (e-mail: zhxiao@hnu.edu.cn; fancheng@hnu.edu.cn; wrh@hnu.edu.cn; hongbojiang2004@gmail.com; yphu@hnu.edu.cn).

Ju Ren is with the School of Computer Science and Engineering, Central South University, Changsha 410083, China (e-mail: renju@csu.edu.cn).

Chenglin Cai is with the College of Information Engineering, Xiangtan University, Xiangtan 411105, China (e-mail: chengcailin@126.com).

Arun Iyengar is with IBM Thomas J. Watson Research Center, Yorktown Heights, NY 10598 USA (e-mail: aruni@us.ibm.com).

Digital Object Identifier 10.1109/JIOT.2020.3001566

I. Introduction

A. Background and Motivation

A VEHICLE trajectory is a path generated by a moving vehicle in space [1], and vehicle trajectory data record the movement of individual vehicles. In recent years, such data have increasingly been important for a wide range of applications, such as driver behavior investigation/classification [2], travel time/distance estimation [3], routing in Internet of Vehicles (IoV) and vehicular *ad hoc* network (VANET) [4], vehicle mobile-edge computing [5], vehicle energy/emission evaluation [6], and social network [7]. Vehicle trajectory can reveal the meaningful structure of a scene, as well as allow inference on the event taking place. For instance, one application is a full view of a vehicle's interactions with others during car following. What is more, a sufficient number of vehicle trajectories can assist in generating underlying digital road maps [8], [9].

Generally, the methods of vehicle trajectory collection can be divided into two categories, i.e., based on videos and mobile sensors, respectively. Traditional vehicle trajectory data extraction is video based and requires substantial data-processing efforts. Also, the task of collecting vehicle trajectories from videos could be challenging and is highly dependent on video quality and algorithms used [10]. In another line, thanks to the technical advances of mobile devices [11], a considerable amount of vehicle trajectory data is available through mobile sensors, such as probe vehicles equipped with smartphones (or mobile devices with GPS modules). For example, floating cars equipped with GPS receivers (around 50 000 taxi cabs and 16 000 buses) in a big city of China transmit their positions to the traffic management center [12].

Research in trajectory-related applications depends on data availability and completeness, especially if the interest is in the individual driver/vehicle behavior modeling and its applications [13]. However, mobile device-extracted data are usually incomplete, which necessitates large-scale deployments along with reliable data collection methods. In particular, one fact we are facing is that the trajectory information of private cars [14], which constitute the majority among urban vehicles (more than 75% in Europe [15], and more than 87% in China [16]), are often hard to collect. The reasons behind this are given are follows (also see detailed explanations in Section II). First, people driving private cars mostly prefer smartphone/car navigation systems. Nevertheless, the smartphone apps developed

by companies, such as Progressive, Cambridge Mobile Telematics, etc. [17], are not suitable for trajectory data collection, which demands an always-on fashion. Second, the car navigation systems are often severely heterogeneous, which makes it hard to provide trajectory data in large-scale deployment scenarios. Concretely speaking, no unified interfaces are available for various vehicle models to enable practical and efficient trajectory data retrieval from a variety of in-car navigation systems. Third, GPS outage is inevitable [18] especially in urban environments, due to the blockage of GPS signal reception and multipath effects. Existing efforts, such as [19]–[21] mainly take advantage of data fusion in a direct manner and do not perform well on complex road sections. Integrating digital maps is a way of helping to reconstruct the trajectory. However, having road information and updating it in a timely fashion incur a considerable deployment cost. Hence, these techniques are not preferred by price-sensitive third-party service providers (or price-sensitive users) [17], [22]. Instead, a low-cost plug-and-play device is more desirable for large-scale deployment of private car trajectory collection.

Summarizing, two challenging issues are remaining when performing large-scale trajectory collection. First, a practical and affordable data collection solution, for instance, using the cheap commercial-off-the-shelf (COTS) devices is preferable for both the vehicle owners and the third-party service providers. Second, the given solution is supposed to be able to achieve a reliable trajectory collection in various urban environments.

B. Contributions

This article presents TrajData, the systematic solution to reliable vehicle trajectory collection which only relies on COTS onboard unit (OBU) devices. These COTS plug-and-play OBU devices are integrated by cheap COTS GPS receivers and low-cost onboard diagnostics (OBD) readers as detailed in Section III. Equipped with the COTS OBU device on vehicles, the vehicle position can be revealed by longitude and latitude coordinates obtained from the external GPS receiver. The OBD reader is plugged into the vehicle OBD interface to read the information from in-vehicle motion sensors, such as velocity and steering direction. Here, no external inertial measurement unit (IMU) device is needed. Besides, the communication unit is employed to transmit the collected trajectory information to the data center.

We emphasize that performing trajectory collection with OBU devices is not straightforward. When driving in various road sections during trajectory collection, the change in road conditions in urban environments causes fluctuation of OBD readings. Thus, the underlying distribution of newly collected data is likely to be different from the previous one, incurring the so-called nonstationary and concept drift problem [23]. Particularly, GPS outage is inevitable in urban environments [24], which degrades the performance of vehicle trajectory collection. To alleviate this suffering, we utilize the motion information retrieved from the OBD reader to help recovering the missing location of trajectory during GPS outage. In addition, GPS outage over a certain period, e.g., larger than 5 s, will incur accumulated errors in trajectory construction. To resolve this, technically, apart from existing efforts [19]–[21] directly using the data-fusion method, we propose a novel data-fusion-enabled deep learning approach to integrate the GPS data with OBD readings, with the purpose on achieving reliable vehicle trajectory collection in various urban road conditions. To specify, we carefully take advantage of the deep learning technique to build up the trajectory reconstruction model, which is based on the knowledge learned from data in a short period before GPS outages. Next, by identifying different road sections via OBD readings, we design a novel trajectory reconstruction method adapting the data fluctuation and error accumulation in a variety of road sections. The main contributions of this article are outlined as follows.

1) We develop TrajData, a systematic solution with reliance only on plug-and-play OBU devices. Our solution combines multisource data of vehicles via making use of cheap COTS GPS modules and low-cost OBD readers, and the interface requirement is very standard. To the best of our knowledge, TrajData is the first work providing a low-cost solution for trajectory collection in large-scale deployment scenarios and is particularly well suitable for private cars. We have applied TrajData into the real-world urban scenario and successfully collected a large data set of private car trajectories.
2) We propose a novel trajectory reconstruction framework, within which the collected trajectory raw data are first input to a classifier so as to distinguish road sections based on the OBD readings. Then, we design a deep learning-based algorithm to realize trajectory recovery during GPS outage based on the knowledge learned from a short period of trajectory data before GPS outage. Furthermore, we integrate the neural arithmetic logic units (NALUs) into our trajectory recovery model so as to address the challenging issues when GPS outage takes place in complex road sections.
3) We conduct real-world road experiments to validate the effectiveness and reliability of TrajData. With reliance only on COTS OBD devices, the TrajData achieves an average position error (PE) below 15-m around a 60-s GPS outage, even in complex road sections, i.e., continuous turns and driving with accelerations/decelerations resulting in frequent changes of direction and speed. Besides, we test our collected trajectory via using TrajData, the results validate that it generates a better road network than state-of-the-art OpenStreetMap (OSM).

The remainder of this article is organized as follows. Section II summarizes the related works. After that, Section III describes the overview of TrajData. Section IV presents the design details of TrajData. Section V shows the experimental evaluation of TrajData. Finally, we conclude this article in Section VI.

II. Related Works

The vehicle trajectory especially the private car trajectory information is often preliminary for many mobile applications [13], [14]. In this context, trajectory information collection from road vehicles has attracted continuous attention from both the research community and industry.

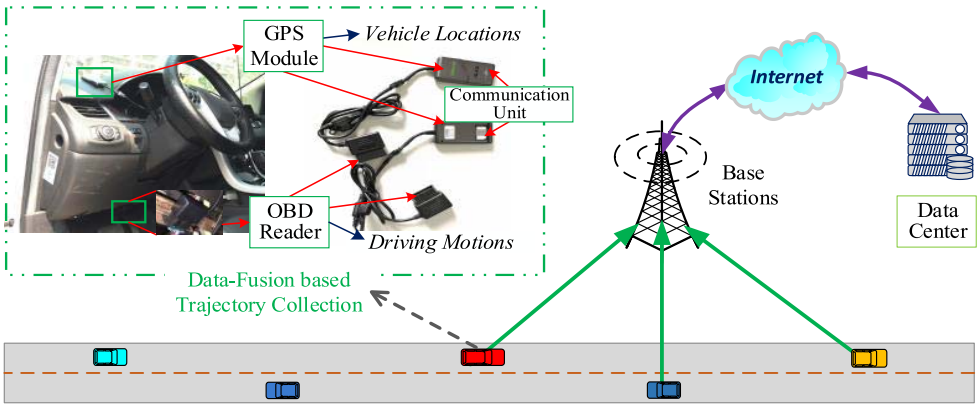


Fig. 1. System implementation of TrajData.

One straightforward solution for trajectory collection is the utilization of smartphones, considering the proliferation of various smart devices such as smartphones wherein the global navigation satellite system (GNSS) and inertial sensors are embedded [25]–[28]. Gustafsson *et al.* [29] presented a comprehensive study on the decennial development of smartphone-based vehicle telematics. Academic and commercial projects are introduced in [29] showing the vehicle trajectory information can be collected via driver's smartphones such as seen in Table II therein. However, it is not suitable for large-scale private car trajectory collection. This conclusion is based on the following observations.

1) In most cases of daily driving, private car users are quite aware of where they are heading and thus do not need smartphone navigation.
2) They may use the smartphone navigation apps for directions when they drive to an unknown location. Note that they need to turn on the app once they start to drive the car. This hinders the feasibility of trajectory collection since manual intervention is required.
3) In addition, the vehicle trajectory data collected via smartphone might be incomplete and discontinuous because in some cases, drivers do not let their smartphone navigation be in use throughout the driving just for saving battery or other purposes. As such, it is not a practical way by means of a smartphone to perform long-playing and large-scale trajectory data collection for private cars.

Another possible solution relies on the in-car navigation for vehicle tracking and trajectory collection. Unfortunately, no unified interfaces are provided for various vehicle models that enable data to be extracted from the in-car navigation system. As for the further comment, only the high-grade cars are equipped from the factory with such an advanced function as in-car navigation. Owners of private cars will choose to, at a reasonable cost, install one of the many third-party in-car navigation systems on the market [30]. In this sense, the in-car navigation is supposed to be referred for an affordable trajectory collection method that needs to be acceptable for many various levels of private car users. In a practical situation, this is often not the case.

Alternatively, many studies strive to collect vehicle trajectory data using external IMU sensors or additional on-vehicle sensors [31]–[34]. They employed external IMU sensors to collect additional vehicle motion information for the standalone GPS, which are vulnerable particularly in the urban area wherein the multipath effect and nonline of sight (NLOS) could lead to GPS outage and hence disable the GPS-based positioning technology. These solutions mainly take advantage of data fusion in a direct manner for trajectory reconstruction and do not perform well on complex road sections such as ramps. Moreover, considering the large-scale use, the high-grade IMU sensors are not conducive to the commercial promotion especially facing the price-sensitive private car users. This inevitably restricts the accuracy of IMU readings and thus degrades the trajectory collection performance.

III. Overview of TrajData

A. System Implementation

On the basis of data fusion, TrajData makes use of COTS OBU devices and exploits the motion sensor information from the OBD reader. As shown in Fig. 1, the hardware of TrajData includes three components: 1) *the GPS module* of a cheap commercial GPS receiver ublox M8030 (it costs $3.5); 2) *the OBD II reader* (it costs $1.0) we design plugging through the OBD interface; and 3) *the communication unit* (it costs $5.0) with a SIM card embedded into the GPS module (it costs total $9.0). The designed lightweight and low-cost OBD II reader is compatible with standard ISO 14230 (KWP2000) and SAE protocols [35]. One point is that in response to the increasing demands of driving safety, anti-lock braking systems (ABSs) and electronic stability programs (ESPs) are becoming standard equipment for automobiles. As such, without using an additional IMU device, the OBD reader, which is connected with a vehicle's OBD interface, is able to read the driving status, such as velocity, acceleration, and steering direction from the in-vehicle motion sensors through the controller area network (CAN) bus [36].

Discussion on the Usage of OBD Reader: Deploying the GPS receiver with external inertial sensors or additional on-vehicle sensors, is an option for implementing trajectory

TABLE I
SAMPLE OF TRIP RECORD IN OUR DATA SET

ObjectID	StartTime	StopTime	StartLon	StartLat	StopLon	StopLat	TripMileage	TripOil	TripPeriod
382704	2018/07/05 09:08:51	2018/07/05 09:31:38	114.047165	22.594368	114.060472	22.535997	7102	983.36	1367

TABLE II
SAMPLE TRAJECTORY AND DRIVING CONDITION RECORD IN THE TRIP

ObjectID	Lon	Lat	GPSStatus	GPSTime	Speed	Direct	Mileage	AlarmDesc	RPM	AccPos
382704	114.047165	22.594368	Strong (9)	2018/07/05 09:08:51	18	150	41991.946	-	951	25.32
382704	114.048513	22.592278	Strong (7)	2018/07/05 09:10:10	14	120	41992.219	-	927	23.14
382704	114.057682	22.578430	Weak (1)	2018/07/05 09:14:55	11	100	41994.083	-	885	19.23

collection. While considering the large-scale use, the external inertial sensors are not conducive to commercial promotion especially facing the price-sensitive vehicle users and service providers. In this line, the OBD reader has a cost advantage when compared with the external inertial sensors. For example, a common IMU device, IMU GY-85 costs around $10, the OBD II is only $1. Apart from the cost consideration, like the inertial sensors, the OBD reader can collect vehicle motion information, including velocity and steering direction. Furthermore, unlike the external inertial sensors, the OBD reader is able to record trajectory trip information, such as trip mileage, travel time of the trip, fuel consumption, and so on. In addition, OBD II provides standard RS-232 interface enabling fast and convenient data transmission to the GPS module via wired connection. Via using the IoV technology, the collected trajectory data are sent back to the data center through the communication unit embedded in the GPS module (see Fig. 1).

The trajectory data collected via TrajData are stored in the data set by individual trips, as shown in Table I. Each trip contains the information of vehicle ID ("ObjectID"), the start and stop time, the start and stop locations (GPS coordinates), the trip mileage in meters ("TripMileage"), the travel time of the trip in seconds ("TripPeriod"), fuel consumption in milliliter ("TripOil"), etc. Besides, as presented in Table II, the driving condition in this trip, such as vehicle locations ("Lon" and "Lat") in line with a timestamp ("GPSTime"), vehicle speed, steering direction ("Direct"), instantaneous cumulative mileage ("Mileage"), revolutions per minute (RPM), accelerator pedal position (AccPos), and description of activated alarms (AlarmDesc), is also included. Here, the *GPSStatus* indicates on a scale of 1–9 how good the GPS signal reception is, namely, whether the GPS is working or if there is an outage. Moreover, we provide two ways via the authentication-required user interface for the private car users to retrieve their own trajectory data using either the smartphone app or by logging into the website to access their trajectory data. Moreover, TrajData collects relatively complete trajectory information, i.e., vehicle position and driving status. Such data can be used for the analysis of driving behaviors and transportation conditions, which fosters a broad range of applications in IoV, usage-based insurance (UBI), social big data analytics, and smart cities.

TrajData starts to collect trajectory data when the vehicle starts up and transmits the data back to the trajectory data center (TDC). It also periodically collects and sends vehicle information with a low data rate when the vehicle has been parked for a long time. The process of trajectory collection and data transmission via TrajData is purely anonymous. Specifically, TrajData is not to collect any personal information regarding the vehicle as well as the driver. The international mobile equipment identity (IMEI) number of the communication unit is assigned as the unique ID for each vehicle and is one-to-one mapped to a bit string as an anonymized ID for privacy protection. The trajectory data are stored in the TDC protected by authentication mechanisms and firewalls in our collaborating third-party service provider's network. For the collection of the trajectory data set, we ensure that the users understand the process of the trajectory collection. In other words, the usage of TrajData is entirely up to users. We have received the approval from the users before we install TrajData on their vehicles. In addition, they gave their consent to educational institutions to study and use the trajectory data for research purposes.

B. Problem Formulation in the Presence of GPS Outages

As stated in Section I, trajectory-related applications are highly dependent on data availability and its completeness. One main obstacle to achieving data completeness is the inevitable GPS outage. GPS outages can lead to missing vehicle locations and large errors in trajectory data. Hence, GPS outages dramatically degrade the performance of trajectory collection and lower the application value of collected trajectory data, especially for fine-grained trajectory data mining. To alleviate this problem, we borrow the idea of data fusion, in the case of this study, utilizing the motion information retrieved from the OBD reader, i.e., vehicle velocity, driving direction, acceleration, etc., to help recovering the missing location of trajectory during GPS outage. We present the problem formation in this section.

In TrajData, the vehicle locations and driving status are obtained via the GPS module and OBD reader, respectively. At time instant t, let $\mathbf{s}_t = (x_t, y_t)$ denote the vehicle position information (the latitude and longitude) collected by a GPS module. a_t^o and ω_t^o represent acceleration and angular speed corresponding to time t, respectively. Note here the acceleration and angular speed can be obtained based on the "AccPos" and "Direct." The OBD reading Direct denotes the direction angle of the vehicle, its value ranges from 0° to 360°.

(a) (b) (c)

Fig. 2. Type I: straight road sections.

This measurement is consistent across vehicles, since the OBD reader records such information from in-vehicle motion sensors through the standard interface. It is noteworthy that the Direct does not explicitly indicate the curvature of the path traveled by the vehicle. As stated in Section IV-A, by investigating the changes of the direction angle, we are able to determine which sections the vehicle drives through. This provides critical information for designing the trajectory reconstruction model.

Let *Ac* and *Dir* denote the OBD readings AccPos and Direct, respectively, we have $a_t^o = Ac_t^o - Ac_{t-1}^o$ and $\omega_t^o = Dir_t^o - Dir_{t-1}^o$.

Given: Observed vehicle position information in the trajectory data $\mathbf{S} = [(x_{T-N+1}, y_{T-N+1}), \ldots, (x_T, y_T), (\tilde{x}_{T+1}, \tilde{y}_{T+1}), \ldots, (\tilde{x}_{T+L}, \tilde{y}_{T+L}), (x_{T+L+1}, y_{T+L+1}), \ldots]$. Known OBD readings $\mathbf{O} = [(a^o_{T-N+1}, \omega^o_{T-N+1}), \ldots, (a^o_{T+L+1}, \omega^o_{T+L+1}), \ldots]$.

Here $[(\tilde{x}_{T+1}, \tilde{y}_{T+1}), \ldots, (\tilde{x}_{T+L}, \tilde{y}_{T+L})]$ denotes the vehicle positions when a GPS outage occurs during trajectory collection. L is used to denote the duration of the GPS outage.

Objective: Reconstruct the trajectory data in the presence of GPS outage, namely, $\mathbf{S}_{rec} = [(\hat{s}_{T+1}, \ldots, \hat{s}_{T+L}] = [(\hat{x}_{T+1}, \hat{y}_{T+1}), \ldots, (\hat{x}_{T+L}, \hat{y}_{T+L})]$.

It should be noted that trajectory prediction or reconstruction is a challenging task when GPS outages occur. The root of this problem stems from the missing GPS measurements which are expected to provide the required vehicle position information during trajectory collection. One widely used solution is to leverage the integration of inertial measurements to provide supplementary data to compensate for the loss of the GPS signal [19], [20], [37]. Besides, the deliberative design for such a data-fusion-based method is required to achieve good results.

To that end, we design a two-step trajectory reconstruction approach in TrajData, in order to infer the vehicle position during trajectory data collection in the case of GPS outage. The aim of the first step (i.e., the training) is to learn the errors of OBD readings corresponding to the trajectory data when GPS is free of the outage. N denotes the size of the training set. When GPS is fully available, the trajectory location obtained via the GPS is sufficiently accurate for many trajectory-related applications, such as driver behavior investigation/classification [2] and travel time/distance estimation [3]. Therefore, we utilize N trajectory points based on the GPS measurements and infer the accurate acceleration a_t^g and angular speed ω_t^g for the vehicle. In addition, by making use of the idea of dead reckoning [38], we can obtain the trajectory location based on the OBD readings a_t^o and ω_t^o, with the help of the initial GPS point (note that at this moment the GPS signal is still available). Because of the inherent noise of in-vehicle motion sensors and more importantly the accumulated errors, the trajectory based on OBD readings is inaccurate (see the results of DR-OBD in Section V-C). To resolve this, in the training phase, we utilize the trajectory location based on GPS to calibrate the errors of the OBD readings. In one word, we learn the gap between (a_t^g, ω_t^g) and (a_t^o, ω_t^o). At time t, the gap is defined as the OBD reading error $\mathbf{e}_t = (e_t^a, e_t^\omega)$, which can be expressed by $e_t^a = a_t^g - a_t^o$ and $e_t^\omega = \omega_t^g - \omega_t^o$.

The second step is to recover the trajectories through a mathematical calculation process based on the relationship between the OBD reading errors and the positions of the vehicle. In fact, we indirectly predict the displacements through the predicted value of the OBD reading errors to achieve the purpose of reconstructing the trajectory. Within the duration of GPS outage $t = T+1, \ldots, T+L$, the trajectory reconstruction process can be expressed as follows:

$$\left(\hat{a}_{t-1}^g, \hat{\omega}_{t-1}^g\right) = \left(a_{t-1}^o, \omega_{t-1}^o\right) + \hat{\mathbf{e}}_{t-1} \tag{1}$$

$$\left(\hat{v}_t^g, \hat{\alpha}_t^g\right) = \left(\hat{v}_{t-1}^g, \hat{\alpha}_{t-1}^g\right) + \left(\hat{a}_{t-1}^g, \hat{\omega}_{t-1}^g\right) \tag{2}$$

where $\hat{v}_t^g$ and $\hat{\alpha}_t^g$ denote the predicted speed and the predicted direction angle of the vehicle at time t, respectively. $\hat{\mathbf{e}}_{t-1}$ represents the estimation value of $\mathbf{e}_{t-1}$.

Accordingly, the predicted displacement between two adjacent trajectory points is calculated by

$$\hat{\mathbf{d}}_t = \left(\hat{v}_t^g * \cos\left(\left(\hat{\alpha}_t^g/180\right) * \pi\right),\ \hat{v}_t^g * \sin\left(\left(\hat{\alpha}_t^g/180\right) * \pi\right)\right). \tag{3}$$

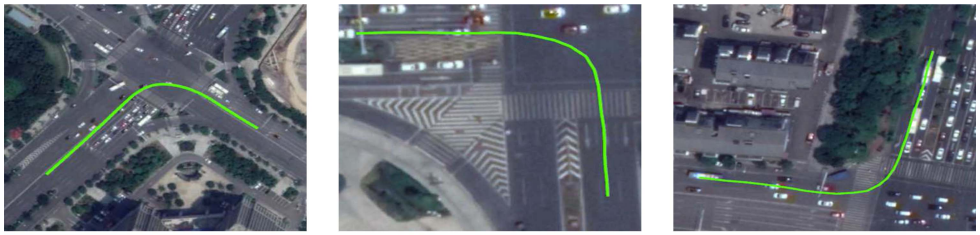
Then, we reconstruct the trajectory locations by

$$\hat{\mathbf{s}}_t = \hat{\mathbf{s}}_{t-1} + \hat{\mathbf{d}}_t. \tag{4}$$

C. Road Section Types

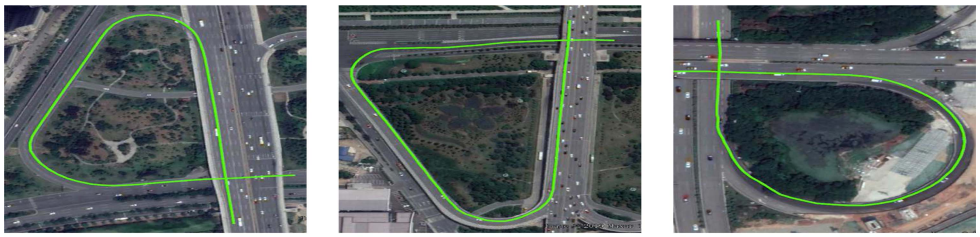
As stated in Section I, our trajectory reconstruction method is able to adapt to different road sections. This is motivated from our observations on road section types. According to our experience on the daily driving, there are generally three types of road sections in urban areas, namely: 1) *Type I:* "straight" sections, including straight roads and slow bends that approximate straight roads, as showed in Fig. 2; 2) *Type II:* right-angle turns in Fig. 3; and 3) *Type III:* ramps, as showed in Fig. 4. Formally, we summarize our observations as follows.

Type I: When people drive along a straight road [see in Fig. 2(a)], they barely change the attitude of the vehicle. As a result, the OBD readings, such as acceleration and steering direction keep almost steady. Even if people occasionally change lanes or pass another vehicle, it would not lead to considerable changes in the vehicle's motion status or steering directions. Note that the road sections in Fig. 2(b) and (c) are also categorized as straight. Indeed, it would require steering relatively large angles, e.g., around 20° when passing the sections in Fig. 2(b) and (c). The curvatures of such road



(a) (b) (c)

Fig. 3. Type II: right-angle turns.



(a) (b) (c)

Fig. 4. Type III: ramps.

Fig. 5. Composite road sections.

sections are rather small, which means people only need to turn gradually, and the direction angle would not be radically altered.

Type II: When driving through right-angle turns as shown in Fig. 3, people need to slow down ready for turning and entering the curve, and then accelerate out of the curve. We observe obvious fluctuations of OBD readings due to the deceleration and acceleration, and the changing of steering direction in these road sections. Besides, we emphasize that driving into and out of the curve is likely straight. However, the duration of the turning part is relatively short compared to the entire right-angle turning. In addition, the radius of turning is small and thereby probably leads to a marked change of the direction angle.

Type III: As illustrated in Fig. 4, ramps play a crucial part in the cloverleaf junction or flyover. When people drive on such road sections, there are two distinguishing features, i.e., low-speed driving and a relatively long time turning. Since ramps are curved, the vehicle attitude changes greatly, thereby resulting in significant changes in driving directions, in the meantime, the OBD reading error increases.

Composite Road Sections: It is noteworthy that there are more complex road sections in urban road networks. Fig. 5 shows examples, which can be regarded as a combination of the straight roads, right-angle turns, and ramps.

Given the above-mentioned observations, we are motivated to utilize the change of direction angle to identify various road sections (see details in Section IV-A).

IV. Design Details of TrajData

A. Road Section Identification

To recover the trajectory locations during outages, we need to determine which sections (i.e., Type I, Type II, or Type III) the vehicle drives through. According to the *observations* in Section III-C, we identify the road sections by investigating the changes of direction angle from the OBD readings during GPS outages. In the following, we select concrete examples of different road sections (see Figs. 2–4), so as to demonstrate the performance of road section identification.

Identify Type I: The direction angles do not change much or change continuously only tens of degrees at a smaller average angular velocity. As shown in Fig. 6, the y-axis presents the range of direction angle and the x-axis denotes the duration of the road section. In Fig. 6, the direction angle changes corresponding to the sections in Fig. 2(a)–(c). It can be seen that the angle of the straight road changes within 5°. When we look into the sections in Fig. 2(b) and (c), the practical driving direction changes in total 20°. The vehicle only needs to slightly and slowly adjust the steering direction when driving

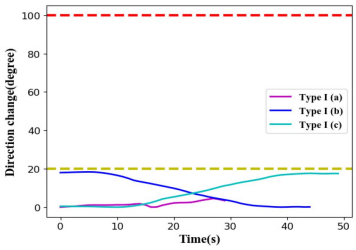


Fig. 6. Direction angle changes in straight roads.

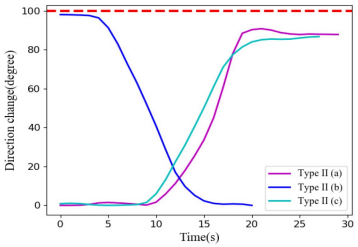


Fig. 7. Direction angle changes in right-angle turns.

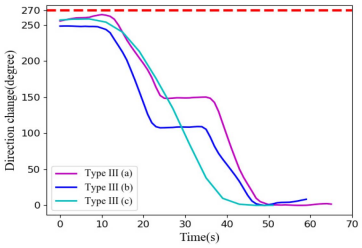


Fig. 8. Direction angle changes in ramps.

along Type II (b) or Type II (c) sections. Therefore, the angle from OBD readings only changes within a small range, e.g., less than 20°, as shown from the results in Fig. 6. In addition, the average angular velocity is less than 1 degree per second.

Identify Type II: The direction angles of the right-angle turning are supposed to be 90°. In addition, the average angular velocity in the turning part should be large. As supported from Fig. 7, the direction angle changes of the vehicle are slightly above or below 90°, and the average angular velocities are above 8 degrees per second in the middle of the sections. In addition, the directions remain almost steady when driving into and out of the curve part of these road sections.

Identify Type III: The direction angles of this road section generally change continuously over a few hundred degrees. As shown in Fig. 8, the steering direction angles of these road sections have continuously changed around 270°. In particular, we observe a straight line between the two curves in the ramp sections as shown in Fig. 4(a) and (b), and a continuous turning in Fig. 4(c). These are in accordance with the results in Fig. 4, namely, two adjacent direction changes with a straight road in the middle as seen in Type III (a) and (b), and continuous direction changes in Type III (c).

When people drive their cars along the road, they produce trajectory and in somehow, the trajectory is related to people's driving behavior. While it is noteworthy that, no matter how people drive, their trajectories are always shaped by the road. As a result, although people drive with their own habit, their trajectories, by and large, follow the shape of the road network. We utilize the direction angle changes, which is sufficient to identify three types of road sections. To specify, for Type I, the attitude of a vehicle on such straight roads remains almost steady, namely, the changes of the direction angle are small, i.e., less than 20°, as seen in Fig. 6. When driving through the right-angle turns (Type II), the direction angles change while not exceed 90°. As for Type III as shown in Fig. 4, which are a crucial part in the cloverleaf junction or flyover (see Fig. 5). In such road sections, the vehicle attitude changes greatly, thereby resulting in significant changes (270° as seen in Fig. 8) in driving directions. Therefore, based on the change of the vehicle's direction angle in three types of road sections, we are able to distinguish road sections via using the OBD readings. In other words, using OBD readings to calculate the change of the vehicle's direction angle provides a reasonable and effective way to identify the road sections.

To conclude, the direction angle obtained from the OBD reader provides a good metric for identifying the road sections, since it changes in line with the different driving behaviors in the three types of road sections. Furthermore, Type I and Type II roads exhibit a strong short-term correlation during trajectory collection. The reasons behind this are that the attitude of a vehicle can remain almost steady on straight roads; the right-angle turns are straight for most of the time, and their turning parts are usually very short. In other words, Type I and Type II roads can be regarded as common road sections. In contrast, ramps are uncommon sections and relatively complex because long-term angular changes and acceleration and deceleration may occur.

B. Trajectory Reconstruction

In principle, the trajectory points are related to previous ones. The motion information of a vehicle during GPS outage is highly correlated to that before the GPS outage occurs. Note that for practical applications, errors and accumulation of errors in the motion information, namely, the OBD readings in this study, inevitably turn out. To resolve this, we strive to learn the characteristics of the errors by designing a training process based on deep neural networks when the GPS signals are available. Moreover, we observe that the motion information, such as the changes of the direction angle retrieved from different road sections exhibit widely different characteristics, which significantly effects the performance of trajectory reconstruction. Considerable changes of the direction angle can occur in complex road sections, e.g., the ramps, which can significantly degrade the accuracy of trajectory reconstruction.

To solve this problem, according to the identification results of road sections, we leverage recurrent neural networks (RNNs) to design the trajectory reconstruction method adapting different road sections with the purpose of achieving reliable trajectory collection in the presence of GPS outages.

RNNs have the ability to succeed on a wide range of challenging prediction problems [39]. They make predictions by using most *raw* data and are thus generally applicable to a wide range of input data. Specifically, in this work, we utilize

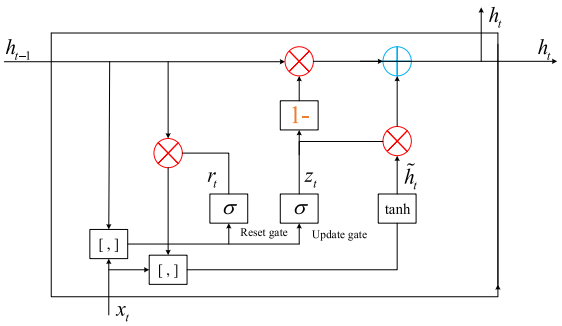


Fig. 9. Internal structure of a GRU cell. σ and tanh, respectively, denote a sigmoid and hyperbolic tangent activation functions; the [,] node operates a weighted concatenation of the new input x_t and the previous output hidden state h_{t-1}.

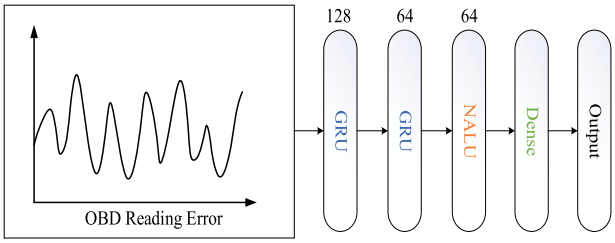


Fig. 10. GRU-NALU structure.

the gated recurrent unit (GRU), a variant of LSTM that also overcomes the problem that RNN does not handle long-term dependencies very well. The GRU has one less gate than the LSTM with fewer parameters, is relatively easy to train, and prevents overfitting. Besides, GRUs have similar performance to LSTM, and sometimes even better. Compared with LSTM, the GRU structure has only two gates, namely, the update gate and the reset gate. Similar to LSTM, GRU has a unique control mechanism that enables it to learn when to reset the past hidden state and update the state when given a new input. Let h_t be the output hidden state at time step t; then h_t is updated by the following equations:

$$r_t = \sigma(W_{xr}x_t + W_{hr}h_{t-1} + b_r) \tag{5}$$
$$z_t = \sigma(W_{xz}x_t + W_{hz}h_{t-1} + b_z) \tag{6}$$
$$\tilde{h}_t = \tanh\left(W_{x\tilde{h}}x_t + W_{rh\tilde{h}}(r_t \odot h_{t-1}) + b_{\tilde{h}}\right) \tag{7}$$
$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t \tag{8}$$

where $\sigma(x) = [1/(1 + e^{-x})]$ is the sigmoid function; x_t, r_t, z_t, and $\tilde{h}_t$ are the input vector, reset gate vector, update gate vector, and hidden state update vector, respectively; W_{xr}, W_{hr}, W_{xz}, W_{hz}, $W_{x\tilde{h}}$, and $W_{rh\tilde{h}}$ are linear transformation matrices; b_r, b_z, and $b_{\tilde{h}}$ are bias vectors; and $\odot$ represents the elementwise production.

Fig. 9 illustrates the internal structure of a GRU cell. The input vector of GRU at each time is the OBD reading error before GPS outage, that is, $x_t = \mathbf{e}_t = (e_t^a, e_t^\omega)$. The output hidden state h_t contains historical OBD reading error information. In the proposed trajectory reconstruction method, we utilize a stacked GRU network, which consists of two GRU layers stacked, one with a layer of 128 cells, and other with a layer of 64 cells, followed by one dense (fully connected) layer containing as many neurons as the number of outputs, in addition to the input layer and output layer. The network is trained using a window of ten time steps, that is, the OBD reading error characteristics of the past 10 s are entered.

Based on the road-type identification, the GRU network structure is adopted for most common road sections, i.e., straight sections (Type I) and right-angle turns (Type II). However, because of its complexity and limitations of the training set, OBD reading errors in ramps are beyond the normal range of the training set, considering that the driving attitude may change over a long period of time in this road section. It can be seen as an *anomaly* change, and traditional DNN structures are incapable of addressing the problem. By reconstructing the basic arithmetic operations, such as addition and multiplication, the NALU can solve such a problem [40]. NALU consists of two neural accumulator (NAC) cells and a sigmoid gate. NALU is capable of systematically learning to represent and manipulate the input vector through primitive arithmetic operations. Let x be the input vector, then the manipulate process of NALU is expressed as follows:

$$a = Wx \tag{9}$$
$$m = \exp W(\log(|x| + \varepsilon)) \tag{10}$$
$$g = \sigma(Gx) \tag{11}$$
$$y = g \odot a + (1 - g) \odot m \tag{12}$$

where W and G are transformation matrices, and g is a learned sigmoidal gate. Equation (9) represents the operation of the first NAC on the input vector, its outputs a are additions or subtractions of rows in the input vector. The second NAC operates in the log space and is therefore capable of learning to multiply and divide, storing its results in m, as shown in (10). Equation (12) represents the manipulation of the input vector of the NALU. Inspired by this, for the complex road section (i.e., Type III), we utilize NALU as the additional unit to our trajectory reconstruction model in order to obtain better generalization both inside and outside of the range of numerical values encountered during training. To that end, we design stacked GRUs combined with the NALU network (GRU-NALU) resulting in an adaptive trajectory reconstruction architecture as presented in Fig. 10. Specifically, we add a NALU layer between the second GRU layer and the dense layer to solve sequence prediction problems in complex road segments. The output hidden states of GRU, which contain historical OBD reading error information, are continuously input to the NALU layer. After that, the NALU gives the network a powerful extrapolation capability to cope with the "abnormal" OBD reading error changes caused by uncommon road sections such as ramps.

Therefore, the mathematical structure of NALU enables it to learn arithmetic functions consisting of multiplication, addition, subtraction, division, and power functions in a way that extrapolates to numbers outside of the range of the output hidden states of GRU containing historical OBD reading errors information. Accordingly, the GRU-NALU is capable of understanding medium-term (up to 10 s) error relations for prediction. The GRU network is trained using a window of

ten time steps, representing 10 s previous observations. This window is updated every second (1 Hz). Considering the short-term correlation during vehicle driving, namely, that driving attitude contained in trajectory data is related to that in the more recent trajectory, training does not require a long trajectory. In our study, we use 20 s trajectory data as the training set, i.e., 20 trajectory points are input as training data when the sampling rate of TrajData is set to 1 Hz. In other words, on each road section when GPS outage takes place, we select 20-s data before the GPS outage for training to recover the trajectory locations. Since our training set is not large, we set the batch size to full batch learning. The parameters of our proposed networks are trained with an *RMSprop* optimizer and the learning rate is set to 0.001.

C. Proposed Trajectory Reconstruction Approach

The framework of the proposed method is presented in Fig. 11. In TrajData, the raw GPS data contains the status of the GPS reception, which indicates whether GPS is accessible during trajectory collection (see the *GPSstatus* in Table II). With the concept of data fusion, when GPS is available, the collected trajectory, including GPS data (e.g., trajectory locations) and OBD readings (e.g., motion information) is input to the training process to learn the error of the OBD readings based on the GPS trajectory locations. To specify, we build up a GRU model to realize trajectory reconstruction during GPS outage based on the knowledge learned from a short period of trajectory data before a GPS outage. When a GPS outage occurs, we utilize the motion information from the OBD readings to distinguish road sections. The road-type information helps to choose the GRU model or the GRU-NALU structure. For the complex road segment (i.e., Type III), we integrate the NALU into the GRU network so as to address the challenging issue when a GPS outage takes place on a ramp.

Recall Section III, the TrajData devices are installed in each individual vehicle and the collected trajectory data are stored in the data center according to the vehicle ID. When looking into the workflow of the proposed method, we find out that the effort on applying the proposed method to different vehicles is moderate. Here, we briefly introduce how the proposed method works. According to the *GPSstatus*, if there is a GPS outage free, the proposed method implements the data fusion as shown in Fig. 11. If GPS outage occurs, we identify the road sections based on the motion information (retrieved from the OBD readings). The road section identification result is input to the data fusion, helping the training in the GRU-NALU structure (see in Section IV-B "trajectory reconstruction"). After that, we obtain the reconstructed trajectory results.

V. Experiments

In this section, the results of the two experiments are highlighted. First, we conduct road tests to evaluate the performance of TrajData and the proposed trajectory reconstruction method via comparisons with existing algorithms. Second, to validate the feasibility of the proposed method, we utilize large-scale vehicle trajectory data, which are collected

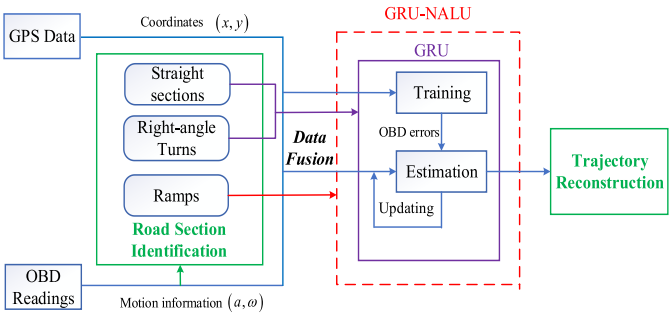


Fig. 11. Proposed framework for trajectory reconstruction.

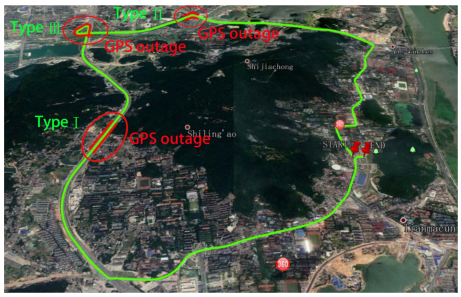


Fig. 12. Experimental route for the road test in Changsha City (courtesy of Google Earth).

via TrajData from real-world urban environments, to generate the road network and compare it with the OSM [8].

A. Alternative Techniques

TrajData-G/TrajData-GN: It is noted that in the procedure of trajectory reconstruction in TrajData, two variants can be used as mentioned in Section IV-B. "TrajData-G" means that the GRU structure acquires knowledge from a short period of trajectory data before GPS outage to recover missing trajectory data, in the network, in the first step of the algorithm, we used two layers of GRU, one of which has 128 units and the other has 64 units. "TrajData-GN" applies the improved GRU-NALU structure, which integrates the NALU into the GRU model on the basis of the results of road section identification, compared to the first step of the TrajData-G algorithm, we added a 64-unit NALU layer after the second GRU layer. In the first step of these two algorithms, the window size of the network is ten time steps, representing 10 s previous observations, and the update frequency of the window is 1 Hz.

RNN: Similar to our proposed TrajData-G method, the neural network structure in the first step of the algorithm is replaced with an ordinary RNN for comparison. RNN has the same parameter setting with TrajData-G. Specifically, in the network, in the first step of the algorithm, we used two layers of SimpleRNN, one of which has 128 units and the other has 64 units. We build up such an RNN model to learn the errors of OBD readings (the training stage) when GPS signals are available and then reconstruct the trajectory (the prediction stage) when GPS outages take place.

DR-OBD: We borrow the concept of dead reckoning [41]. The dead reckoning method used in the experiments is the same as the second step of the dead reckoning process

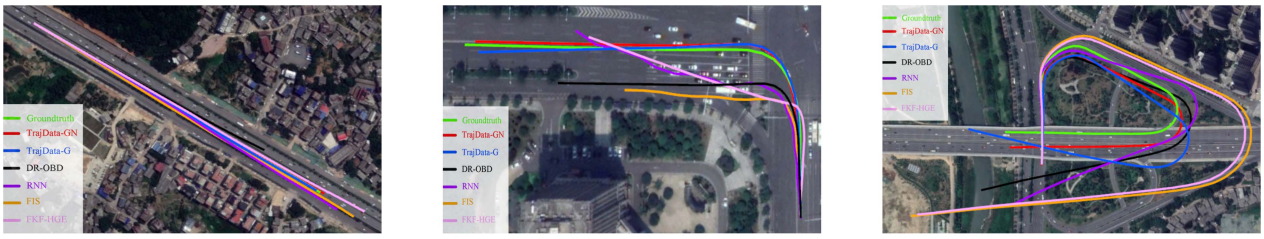


Fig. 13. Trajectory reconstruction in different road sections with GPS outage. (a) Straight road (Type II). (b) Right-angle turn (Type II). (c) Ramp (Type III).

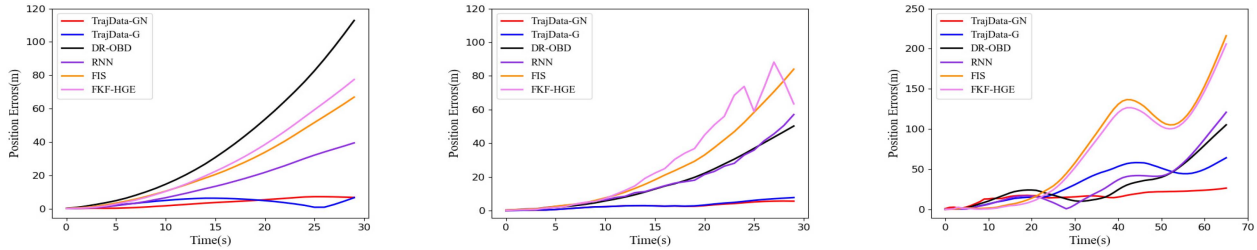


Fig. 14. PEs in different road sections with GPS outage. (a) Straight road (Type II). (b) Right-angle turn (Type II). (c) Ramp (Type III).

of the proposed algorithm, as shown in (1)–(4). The trajectory sampling interval is set to 1 s, that is, the time interval between two dead reckoning points is 1 s. Note that we do not introduce any external IMU. The OBD readings, namely, the vehicle motion information is used as the data required in the processing of dead reckoning. This method is hereafter referred to as "DR-OBD."

FKF-HGE: According to [20], a hybrid global estimator (HGE) is constructed by augmenting the FKF with a gray predictor for vehicle positioning. Suggested from [20], we adopt GM(2, 1) to develop three gray predictors, which are designed in parallel for the PEs along latitude, longitude, and altitude, and the HGE works at 1 Hz.

FIS: Based on [19], the GPS outages are defined by adjusting the GPS propagation weight by a Gaussian model and updated by the fuzzy logic system. The size of the sliding window, $\beta = (\vartheta - \tau)K$, is to control the data flow generated by both OBD and GPS, which is set to 1.254. According to [19], the other parameters are $\vartheta = 1$ and $\tau = 0.373$. $K = 2$ stands for the two navigation systems, namely, GPS and OBD.

B. Road Test

To validate the performance of the proposed trajectory reconstruction approach, we conduct real-world road tests for trajectory collection. Fig. 12 depicts the road test trajectory, which is mainly located in an urban area and includes three types of road sections (see descriptions in Section III-C), such as straight road, left-angle turn, and ramp. Specifically, we drive the test vehicle, an SUV Ford EDGE 2016, by installing TrajData as illustrated in Fig. 1 and obtaining the trajectory from Fig. 12. The sampling rate in the trajectory collection is 1 Hz. In TrajData, the COTS GPS module (i.e., ublox M8030) is typically accurate to 15 m when the vehicle is driving through an open area with a good GPS signal.

C. Analysis of Results

Fig. 13(a) presents the trajectory reconstruction results in the straight road section, in which the length of the GPS outage is 30 s. The trajectory with green line represents the true positions when the test vehicle is driving through the straight road (namely, the ground truth); other lines give results which are obtained by applying, respectively, the TrajData-GN, TrajData-G, DR-OBD, FKF-HGE, FIS, and RNN based on the originally collected trajectory data.

As indicated in Fig. 13(a), apart from the errors becoming large when a GPS outage lasts long, the comparative methods, such as FKF-HGE, FIS, and RNN can recover the good straight trajectory. The reason behind this is that the steering directions from OBD readings remain almost steady during a GPS outage in this road section; these methods can work well at the beginning of the GPS outage. Furthermore, TrajData-G and TrajData-GN produce more accurate trajectories than the other methods, since they nearly perfectly match the true trajectory. Fig. 14(a) presents trajectory PEs of proposed approaches and comparative methods during a GPS outage. Large PEs are seen in the results of DR-OBD (the black curve), since the motion information (e.g., the acceleration) from in-vehicle motion sensors is unable to precisely track the vehicle's motion due to the inherent noise and error accumulation. Besides, it is shown that PE is growing during GPS outage, and the TrajData-G/TrajData-GN keeps PEs relatively low, outperforming the other methods.

Figs. 13(b) and 14(b) present the results from the right-angle turn section. The GPS outage is 30 s. When driving to enter the left-angle turn, the vehicle moves with low speed and accelerations/decelerations. In this road section, the information of acceleration and directions from the motion sensors are inaccurate, this leads to the outputs of DR-OBD and FIS are shorter than the true trajectory and the predicted trajectories tend to

TABLE III
AVERAGE TRAJECTORY RECONSTRUCTION ERRORS (RMSE)

	TrajData-GN	TrajData-G	DR-OBD	RNN	FIS	FKF-HGE
Type I	4.42	4.264	52.571	20.568	30.635	35.5
Type II	3.24	3.825	25.2	24.284	36.142	40.25
Type III	15.3	35.634	38.258	41.562	76.452	75.456

drift away from the true one. FKF-HGE and RNN can partially address the nonstationary problem, while they fail to determine the errors of motion information. As a result, large PEs in these methods result from data fluctuation and error accumulation during trajectory collection in this road section. In particular, these methods are unable to recover properly from the turning part, as shown in Fig. 14(b). Obvious bias occurs in the turning part for the results of FKF-HGE and RNN. Thanks to the incremental learning ability, TrajData-G and TrajData-GN can recover the trajectory close to the true one, and TrajData-GN provides better results, since they are capable of mitigating the error accumulation of the OBD data and yield lower PEs both in latitude and longitude than other methods.

Figs. 13(c) and 14(c) present results from the ramp section. The GPS outage is 66 s. The vehicles drive with low average speed and keep turning. This significantly affects the performance of in-vehicle motion sensors. Besides, the driving attitude remains changing in ramps, which is completely different from that before entering such a road section. In other words, this specific driving status, i.e., keep turning for more than 40 s, seldom occurs in the training set. This hinders the system from acquiring sufficient knowledge from the training stage, namely, a certain period (e.g., 20 s) with no GPS outage. That is to say, the motion information obtained from the OBD reader is not stable and may not be capable of providing adequate information when GPS outages occur in this road section. Accordingly, as shown in Fig. 13(c), there are large trajectory errors when applying FKF-HGE, FIS, and RNN. In this case, even TrajData-G is unable to work properly because the traditional DNN structures are incapable of addressing the problem, which is the input data lying outside of the numerical range during training. By reconstructing the basic arithmetic operations, NALU can well solve the data-extra problem. As a result, the TrajData-GN achieves the best trajectory reconstruction performance and poses less trajectory drift in comparison to all of the other methods. In addition, the results of PE from Fig. 14(c) demonstrate that TrajData-GN only produces small and slow-growth accumulative PEs with longer GPS outages.

Fig. 15 presents the CDF of PEs of TrajData in three types of road sections. Compared to Type III, we achieve better trajectory reconstruction results in road sections Type I and Type II. This is also supported from the results in Figs. 13 and 14. The reasons behind this are that both the straight road sections and right-angle turns have short-term correlation. The vehicle attitude barely changes when driving through a Type I road section, and the time of the turning part in the entire right-angle turns is short even though the change of the direction angle approaches 90° in this road section. Besides, in the ramps' sections, considerable changing of vehicle attitude can be observed. Hence, it is difficult to

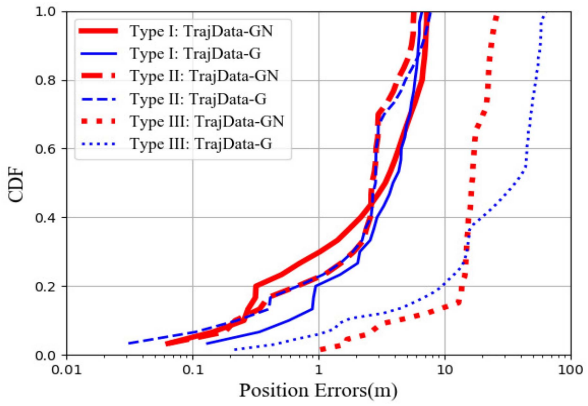


Fig. 15. CDF of PEs of TrajData with GRU-NALU/GRU in three types of road sections.

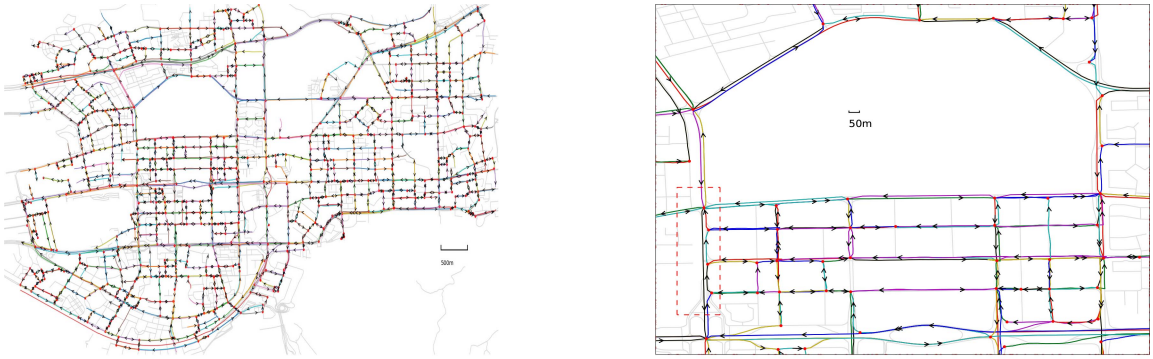
learn the internal connection from this "uncommon" situation. Thanks to the ability for solving such data-extra problems, the NALU integrated with GRU, namely, TrajData-GN, outperforms TrajData-G, especially in ramps.

Furthermore, we select a test set from our data set in Changsha City, which contains 76 trips making several test routes. We calculate the average trajectory reconstruction errors, i.e., the average root-mean-square errors (RMSEs), as presented in Table III. The results demonstrate that our proposed method achieves much smaller average errors than the comparative methods, especially for the complex road section, the proposed method obtains errors around 15 m when driving on relatively complex road segments (i.e., continuous turns and driving with accelerations/decelerations resulting in a frequent change of direction and speed) with long-term GPS outage.

D. Road Network Generation From the Collected Trajectory Data

As mentioned in Section I, a considerable number of vehicle trajectories can help to generate underlying digital road routable maps (the road network). TrajData has been successfully applied to real-world trajectory data collection. In Section III-A, Tables I and II present examples of trajectory trips and records. To validate its effectiveness on road network generation, in this section, we use a fraction of the collected trajectories to construct the road network and compare it with the widely used OSM [8].

To that end, we select a two-week trajectory data set collected from 1345 cars, which contains 20 000 individual trips and covers an urban area encompassing 63 km^2. The road network generation algorithm based on this trajectory data set can be briefly given as follows [42].



(a) (b)

Fig. 16. (a) Road networks. (b) Details (zoomed in).

1) Calibrate vehicle locations according to the trajectory points of trips and determine the driving directions which are used to determine the lane direction, e.g., the two-way road or one-way road.
2) Find out the major and minor intersections based on the road sections' information.
3) Detect the road segments and then build up the road network.

Fig. 16(a) depicts the road networks generated by using these trajectory trips. The road segments are represented by color lines with driving directions (see the arrows), and the road segments from the OSM map are represented with dark gray solid lines. Fig. 16(b) zooms in for more details. The trajectory data in this experiment can be found in https://github.com/HunanUniversityZhuXiao/PrivateCarTrajectoryData.

The accuracy of road network generation is highly dependent on the information of intersections that normally contain right-angle turns and ramps. The OSM is built up in a collaborative manner or a crowdsourcing-alike way [8], namely, people (volunteers) can upload trajectory points, which are mainly obtained from portable or onboard GPS devices, when they pass through the road. Then, the road networks can be constructed based on the integration of the trajectories from many volunteers. However, the OSM-based road network would be inaccurate if GPS outages take place since, in such situations, the trajectory location could be lost. For example, as seen in the rectangular box marked with a red dotted line in the bottom left of Fig. 16(b), various types of road sections are included in this box, and GPS outages have been confirmed in the trajectory collection because of signal blocking by the tall buildings nearby and the multipath effect. Our method can achieve better performance of trajectory reconstruction in the case of GPS outage and hence provides precise accurate trajectory information, especially for the complex road sections. As a result, the trajectory trips collected by our TrajData are able to produce better road network maps that are more accurate than the one from OSM, particularly for complex road intersections and segments. Compared with the real road network, the reconstructed road section error is rather small, which proves that our method has good applicability in various types of vehicles.

VI. Conclusion

In this article, we developed TrajData with reliance only on the COTS OBU devices having the lightweight GPS module and the low-cost OBD reader. It is a feasible and practical solution to realize vehicle trajectory data collection especially suitable for large-scale deployments such as the ever-increasing of private cars. To cope with the GPS outage and concept drift problem in the processing of trajectory collection, we proposed a novel data-fusion-based approach, which integrates the GPS data with OBD readings to build up the deep learning-enabled trajectory reconstruction model adapting the data fluctuation and error. For the evaluation, the road test demonstrates that the proposed data-fusion method outperforms the existing methods in terms of accuracy and reliability in various complex road sections in the presence of GPS outage. Besides, based on our collected trajectory data, the results validate that it can obtain a better road network compared with the widely used OSM.

In practical use, the users are willing to install the TrajData and pay for the third-party service which is based on the trajectory data analysis. It also raises technical issues, for instance, the massive trajectory database poses an unaffordable burden for transmission overhead and data storage. Next, we will devote our effort to investigating the trajectory compression and conducting trajectory big data mining based on the data set collected via TrajData.

References

[1] S. Kuutti *et al.*, "A survey of the state-of-the-art localization techniques and their potentials for autonomous vehicle applications," *IEEE Internet Things J.*, vol. 5, no. 2, pp. 829–846, Apr. 2018.

[2] Q. Xue, K. Wang, J. J. Lu, and Y. Liu, "Rapid driving style recognition in car-following using machine learning and vehicle trajectory data," *J. Adv. Transp.*, vol. 2, Jan. 2019, Art. no. 9085238.

[3] J. Fan, C. Fua, K. Stewarta, and L. Zhang, "Using big GPS trajectory data analytics for vehicle miles traveled estimation," *Transp. Res. C Emerg. Technol.*, vol. 101, pp. 298–307, Jun. 2019.

[4] K. Lin, C. Li, G. Fortino, and J. J. P. C. Rodrigues, "Vehicle route selection based on game evolution in social Internet of Vehicles," *IEEE Internet Things J.*, vol. 5, no. 4, pp. 2423–2430, Aug. 2018.

[5] Z. Xiao *et al.*, "Vehicular task offloading via heat-aware MEC cooperation using game-theoretic method," *IEEE Internet Things J.*, vol. 7, no. 3, pp. 2038–2052, Mar. 2020.

[6] X. Shan, P. Hao, X. Chen, K. Boriboonsomsin, G. Wu, and M. J. Barth, "Vehicle energy/emissions estimation based on vehicle trajectory reconstruction using sparse mobile sensor data," *IEEE Trans. Intell. Transp. Syst.*, vol. 20, no. 2, pp. 716–726, May 2018.

[7] J. Li *et al.*, "Drive2friends: Inferring social relationships from individual vehicle mobility data," *IEEE Internet Things J.*, vol. 7, no. 6, pp. 5116–5127, Jun. 2020.

[8] *OpenStreetMap!* Accessed: Dec. 2, 2019. [Online]. Available: https://www.openstreetmap.org/

[9] X. Liu, J. Biagioni, J. Eriksson, Y. Wang, G. Forman, and Y. Zhu, "Mining large-scale, sparse GPs traces for map inference: Comparison of approaches," in *Proc. ACM SIGKDD*, 2012, pp. 669–677.

[10] Y. Miao, X. Tao, X. Xu, and J. Lu, "Joint 3-D shape estimation and landmark localization from monocular cameras of intelligent vehicles," *IEEE Internet Things J.*, vol. 6, no. 1, pp. 15–25, Feb. 2019.

[11] H. Chen, B. Guo, Z. Yu, and Q. Han, "CrowdTracking: Real-time vehicle tracking through mobile crowdsensing," *IEEE Internet Things J.*, vol. 6, no. 5, pp. 7570–7583, Oct. 2019.

[12] X. Shan, P. Hao, X. Chen, K. Boriboonsomsin, G. Wu, and M. J. Barth, "Probabilistic model for vehicle trajectories reconstruction using sparse mobile sensor data on freeways," in *Proc. IEEE 19th Int. Conf. Intell. Transp. Syst.*, 2016, pp. 689–694.

[13] Y. Huang, Z. Xiao, D. Wang, H. Jiang, and D. Wu, "Exploring individual travel patterns across private car trajectory data," *IEEE Trans. Intell. Transp. Syst.*, early access, Oct. 23, 2019, doi: 10.1109/TITS.2019.2948188.

[14] D. Wang *et al.*, "Stop-and-wait: Discover aggregation effect based on private car trajectory data," *IEEE Trans. Intell. Transp. Syst.*, vol. 20, no. 10, pp. 3623–3633, Oct. 2019.

[15] L. Mantzos, P. Capros, N. Kouvaritakis, and M. Zeka-Paschou, *European Energy and Transport: Trends to 2030*. Brussels, Belgium: Directorate Gen. Energy Transp., Jan. 2003.

[16] C. N. S. Bureau, *China Statistical Yearbook*. Beijing, China: China Stat., 2017.

[17] J. Wahlstrom, I. Skog, and P. Handel, "Smartphone-based vehicle telematics: A ten-year anniversary," *IEEE Trans. Intell. Transp. Syst.*, vol. 18, no. 10, pp. 2802–2825, Apr. 2017.

[18] Z. Xiao *et al.*, "Toward accurate vehicle state estimation under non-Gaussian noises," *IEEE Internet Things J.*, vol. 6, no. 6, pp. 10652–10664, Dec. 2019.

[19] V. Havyarimana, D. Hanyurwimfura, P. Nsengiyumva, and X. Xiao, "A novel hybrid approach based-SRG model for vehicle position prediction in multi-GPS outage conditions," *Inf. Fusion*, vol. 41, pp. 1–8, May 2018.

[20] X. Li and Q. Xu, "A reliable fusion positioning strategy for land vehicles in GPS-denied environments based on low-cost sensors," *IEEE Trans. Ind. Electron.*, vol. 64, no. 4, pp. 3205–3215, Apr. 2017.

[21] X. Li, W. Chen, C. Chan, B. Li, and X. Song, "Multi-sensor fusion methodology for enhanced land vehicle positioning," *Inf. Fusion*, vol. 46, pp. 51–62, Mar. 2019.

[22] G. Castignani, T. Derrmann, R. Frank, and T. Engel, "Driver behavior profiling using smartphones: A low-cost platform for driver monitoring," *IEEE Intell. Transp. Syst. Mag.*, vol. 7, no. 1, pp. 91–102, Jan. 2015.

[23] J. Xiao, Z. Xiao, D. Wang, J. Bai, V. Havyarimana, and F. Zeng, "Short-term traffic volume prediction by ensemble learning in concept drifting environments," *Knowl. Based Syst.*, vol. 164, pp. 213–225, Jan. 2019.

[24] J. Li, N. Song, G. Yang, M. Li, and Q. Cai, "Improving positioning accuracy of vehicular navigation system during GPS outages utilizing ensemble learning algorithm," *Inf. Fusion*, vol. 35, pp. 1–10, May 2017.

[25] C. Barrios, Y. Motai, and D. Huston, "Trajectory estimations using smartphones," *IEEE Trans. Ind. Electron.*, vol. 62, no. 12, pp. 7901–7910, Dec. 2015.

[26] Z. Wu, J. Li, J. Yu, Y. Zhu, G. Xue, and M. Li, "L3: Sensing driving conditions for vehicle lane-level localization on highways," in *Proc. IEEE INFOCOM*, 2016, pp. 1–9.

[27] D. Chen, K.-T. Cho, S. Han, Z. Jin, and G. S. Kang, "Invisible sensing of vehicle steering with smartphones," in *Proc. Int. Conf. Mobile Syst. Appl. Services*, 2015, pp. 1–13.

[28] K. W. Chiang, G. J. Tsai, H. W. Chang, C. Joly, and N. EI-Sheimy, "Seamless navigation and mapping using an INS/GNSS/grid-based SLAM semi-tightly coupled integration scheme," *Inf. Fusion*, vol. 50, pp. 181–196, Oct. 2019.

[29] F. Gustafsson, N. Bergman, U. Forssell, J. Jansson, R. Karlsson, and P.-J. Nordlund, "Particle filters for positioning, navigation, and tracking," *IEEE Trans. Signal Process.*, vol. 50, no. 2, pp. 425–437, Feb. 2002.

[30] I. Skog and P. Handel, "In-car positioning and navigation technologies—A survey," *IEEE Trans. Intell. Transp. Syst.*, vol. 10, no. 1, pp. 4–21, Mar. 2009.

[31] S. J. Kyu, J. Jeungin, M. H. Gi, and J. Daehong, "Sensor fusion-based low-cost vehicle localization system for complex urban environments," *IEEE Trans. Intell. Transp. Syst.*, vol. 18, no. 5, pp. 1078–1086, Aug. 2016.

[32] E. Choi and S. Chang, "A consumer tracking estimator for vehicles in GPS-free environments," *IEEE Trans. Consum. Electron.*, vol. 63, no. 4, pp. 450–458, Nov. 2017.

[33] R. Liu, C. Yuen, T.-N. Do, M. Zhang, Y. L. Guan, and U.-X. Tan, "Cooperative positioning for emergency responders using self IMU and peer-to-peer radios measurements," *Inf. Fusion*, vol. 56, pp. 93–102, Apr. 2020.

[34] C. Melendez-Pastor, R. Ruiz-Gonzalez, and J. Gomez-Gil, "A data fusion system of GNSs data and on-vehicle sensors data for improving car positioning precision in urban environments," *Expert Syst. Appl.*, vol. 80, no. 1, pp. 28–38, Sep. 2017.

[35] *Protocols for Programming Interfaces*. Accessed: Aug. 18, 2019. [Online]. Available: https://automotive.softing.com/en/standards/protocols.html

[36] F. Baronti *et al.*, "Design and verification of hardware building blocks for high-speed and fault-tolerant in-vehicle networks," *IEEE Trans. Ind. Electron.*, vol. 58, no. 3, pp. 792–801, Apr. 2011.

[37] L. N. Balico, A. A. F. Loureiro, E. F. Nakamura, R. S. Barreto, R. W. Pazzi, and H. A. B. F. Oliveira, "Localization prediction in vehicular ad hoc networks," *IEEE Commun. Surveys Tuts.*, vol. 20, no. 4, pp. 2784–2803, 4th Quart., 2018.

[38] Y. Wu, H. Zhu, Q. Du, and S. Tang, "A survey of the research status of pedestrian dead reckoning systems based on inertial sensors," *Int. J. Autom. Comput.*, vol. 16, no. 1, pp. 67–85, 2019.

[39] Z. Yang, X. Guo, Z. Chen, Y. Huang, and Y. Zhang, "RNN-Stega: Linguistic steganography based on recurrent neural networks," *IEEE Trans. Inf. Forensics Security*, vol. 14, no. 5, pp. 1280–1295, May 2019.

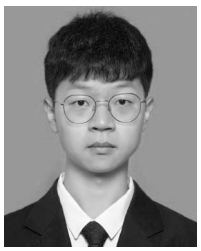
[40] A. Trask, F. Hill, S. Reed, J. Rae, C. Dyer, and P. Blunsom, "Neural arithmetic logic units," in *Proc. Adv. Neural Inf. Process. Syst.*, Aug. 2018, pp. 8046–8055.

[41] A. A. Panyov, A. A. Golovan, and A. S. Smirnov, "Indoor positioning using Wi-Fi fingerprinting pedestrian dead reckoning and aided INS," in *Proc. Int. Symp. Inertial Sensors Syst. (ISISS)*, Laguna Beach, CA, USA, 2014, pp. 1–2.

[42] Y. Huang, Z. Xiao, X. Yu, D. Wang, V. Havyarimana, and J. Bai, "Road network construction with complex intersections based on sparsely-sampledprivate car trajectory data," *ACM Trans. Knowl. Disc. Data*, vol. 13, no. 3, pp. 1–28, Jun. 2019.

Zhu Xiao (Senior Member, IEEE) received the M.S. and Ph.D. degrees in communication and information system from Xidian University, Xi'an, China, in 2007 and 2009, respectively.

From 2010 to 2012, he was a Research Fellow with the Department of Computer Science and Technology, University of Bedfordshire, Luton, U.K. He is currently an Associate Professor with the College of Computer Science and Electronic Engineering, Hunan University, Changsha, China. His research interests include mobile communications, wireless localization, Internet of Vehicles, and mobile computing.



Fancheng Li received the B.S. degree from the Hunan University of Science and Technology, Xiangtan, China, in 2018. He is currently pursuing the M.S. degree in computer science and technology with Hunan University, Changsha, China.

His research interests include heterogeneous networks and wireless networks.

Ronghui Wu received the M.S. and Ph.D. degrees in computer science from Hunan University, Changsha, China, in 2000 and 2010, respectively.

She is currently an Associate Professor with Hunan University. Her main research interests include network test and performance evaluation, wireless communications, and mobile computing.

Hongbo Jiang (Senior Member, IEEE) received the Ph.D. degree from Case Western Reserve University, Cleveland, OH, USA, in 2008.

He is currently a Full Professor with the College of Computer Science and Electronic Engineering, Hunan University, Changsha, China. He ever was a Professor with the Huazhong University of Science and Technology, Wuhan, China. His research concerns computer networking, especially algorithms and protocols for wireless and mobile networks.

Prof. Jiang is serving as an Editor for the IEEE/ACM TRANSACTIONS ON NETWORKING, an Associate Editor for the IEEE TRANSACTIONS ON MOBILE COMPUTING, and an Associate Technical Editor for *IEEE Communications Magazine*.

Yupeng Hu received the M.S. and Ph.D. degrees in computer science from Hunan University, Changsha, China, in 2005 and 2008, respectively.

He is currently a Professor with the College of Computer Science and Electronic Engineering, Hunan University, where he is the Dean of the Department of Cyberspace Security. He was a Visiting Scholar with the Department of Computer Science and Engineering, UT-Arlington, Arlington, TX, USA, from 2015 to 2016. He was as an Academic Visitor with IBM China Development Laboratory, Beijing, China, in 2012. His research interests include storage systems, erasure coding, and network security.

Prof. Hu is a Senior Member of ACM.

Ju Ren (Senior Member, IEEE) received the B.Sc., M.Sc., and Ph.D. degrees in computer science from Central South University, Changsha, China, in 2009, 2012, and 2016, respectively.

From 2013 to 2015, he was a visiting Ph.D. student with the Department of Electrical and Computer Engineering, University of Waterloo, Waterloo, ON, Canada. He is currently a Professor with the School of Computer Science and Engineering, Central South University. His research interests include Internet of Things, wireless communication, network computing, and cloud computing.

Prof. Ren is a Senior Member of ACM.

Chenglin Cai received the M.S. and Ph.D. degrees from the National Time Service Center, Chinese Academy of Sciences, Beijing, China, in 2001 and 2009, respectively.

He is currently a Professor with the School of Automation and Electronic Information, Xiangtan University, Xiangtan, China. His research interests include positioning method and theory of the satellite navigation system, and wireless communication system.

Arun Iyengar (Fellow, IEEE) received the M.S. and Ph.D. degrees in computer science from MIT, Cambridge, MA, USA, in 1988 and 1992, respectively.

He was a National Science Foundation Graduate Fellow with MIT. He does work in distributed computing, cloud computing, and artificial intelligence with IBM Research, Yorktown Heights, NY, USA. His techniques in caching and load balancing, and serving dynamic content are widely used in Web and other distributed applications.

Dr. Iyengar won several best paper awards, has received the IFIP Silver Core Award, and has been named as an IBM Master Inventor multiple times. He served as a Founding Co-Editor-in-Chief for the *ACM Transactions on the Web*. He is currently an Executive Committee Member and a former Chair for the IEEE Computer Society's Technical Committee on the Internet, and an Editorial Board Member for the IEEE TRANSACTIONS ON CLOUD COMPUTING and IEEE INTERNET COMPUTING.